

مبانی و الگوریتم‌های کلان‌داده

جزوه جامع درسی و مرجع دانشگاهی تحلیل کلان‌داده

بر اساس سرفصل‌ها و اسلایدهای کتاب (MMDS) Datasets Massive of Mining
تألیف: ژوره لسکوک، آناند راجارامان، جفری اولمن

گردآورنده: احسان شهبازی

مجموعه یادداشتهای آموزشی کارشناسی ارشد

۷ مهر ۱۴۰۵

فصل ۶

مجموعه‌آیتم‌های پرتکرار و قوانین انجمنی

۱.۶ مقدمه و مدل سبد بازار (Market-Basket Model)

یکی از کلاسیک‌ترین و در عین حال پرکاربردترین الگوها در کلان‌داده، کشف وابستگی‌ها میان اقلامی است که معمولاً به صورت هم‌زمان رخ می‌دهند. این چارچوب تحلیلی تحت عنوان **مدل سبد بازار**^۱ شناخته می‌شود. در مدل سبد بازار:

- مجموعه‌ای بزرگ از آیتم‌ها^۲ وجود دارد (مانند کالاها در فروشگاه آمازون).
 - مجموعه‌ای بسیار بزرگ از **سبدها**^۳ وجود دارد که هر سبد، زیرمجموعه‌ای کوچک از آیتم‌ها است (مانند سبد خرید یک مشتری در یک بار مراجعه).
 - فرض بر این است که تعداد کل آیتم‌ها ممکن است در حد میلیون‌ها، و تعداد سبدها در حد صدها میلیون باشد، اما تعداد آیتم‌های درون هر سبد معمولاً اندک (مثلاً کمتر از ۲۰ قلم) است.
- کاربردهای این مدل فراتر از خرده‌فروشی است:
- **کاوش مفاهیم و کلمات هم‌نشین:** آیتم‌ها کلمات و سبدها اسناد متنی هستند (کشف عبارات مرکب).
 - **زیست‌انفورماتیک:** آیتم‌ها نشانگرهای زیستی و سبدها پرونده ژنتیکی بیماران مبتلا به یک بیماری خاص هستند.
 - **تحلیل سرقت ادبی:** آیتم‌ها جملات و سبدها مقالات هستند.

۲.۶ تعاریف پایه: پشتیبانی، اطمینان و قوانین انجمنی

تعریف ۱.۶: پشتیبانی یک مجموعه‌آیتم

پشتیبانی^a یک مجموعه‌آیتم I که با $s(I)$ نشان داده می‌شود، برابر است با تعداد سبدهایی که I زیرمجموعه‌ای از آن‌ها است:

$$s(I) = |\{B \in \text{Baskets} : I \subseteq B\}| \quad (1.6)$$

اگر یک **آستانه پشتیبانی**^b مانند s داشته باشیم، هر مجموعه‌آیتم I که شرط $s(I) \geq s$ را ارضا کند، یک **مجموعه‌آیتم پرتکرار**^c نامیده می‌شود.

Support^a
Support Threshold^b
Frequent Itemset^c

Market-Basket Model^۱
Items^۲
Baskets^۳

تعریف ۲.۶: قوانین انجمنی، اطمینان و بهره

یک **قانون انجمنی**^a به فرم $I \rightarrow j$ بیان می‌شود؛ یعنی «اگر سبدی شامل مجموعه‌آیتم I باشد، آنگاه این سبد با احتمال بالایی شامل آیتم j نیز خواهد بود».

• **اطمینان (Confidence):** نسبت احتمال وقوع توأم به احتمال وقوع شرط:

$$\text{conf}(I \rightarrow j) = \frac{s(I \cup \{j\})}{s(I)} \quad (۲.۶)$$

• **بهره (Interest):** تفاوت میان اطمینان قانون و احتمال وقوع تصادفی آیتم j در کل سبدها:

$$\text{interest}(I \rightarrow j) = \text{conf}(I \rightarrow j) - P(j) = \text{conf}(I \rightarrow j) - \frac{s(\{j\})}{|Baskets|} \quad (۳.۶)$$

قوانینی جذاب و ارزشمند هستند که بهره آن‌ها مقدار مثبت بزرگی داشته باشد (همبستگی مثبت قوی).

Association Rule^a

مثال کاربردی ۱.۶: محاسبه اطمینان و بهره

فرض کنید در یک فروشگاه آنلاین با ۱۰,۰۰۰ سبد:

• سبدهای حاوی پوشک: ۱۰۰۰ $s(\text{پوشک}) = ۰.۱$ $(P(\text{پوشک}))$.

• سبدهای حاوی شیر خشک: ۲۰۰۰ $s(\text{شیر}) = ۰.۲$ $(P(\text{شیر}))$.

• سبدهای حاوی هر دو: ۸۰۰ $s(\{\text{پوشک}, \text{شیر}\})$.

برای قانون شیر $\rightarrow$ پوشک:

$$\text{conf}(\text{شیر} \rightarrow \text{پوشک}) = \frac{۸۰۰}{۱۰۰۰} = ۰.۸ \quad (۸۰\%) \quad (۴.۶)$$

$$\text{interest}(\text{شیر} \rightarrow \text{پوشک}) = ۰.۸ - ۰.۲ = +۰.۶ \quad (۵.۶)$$

چون بهره مثبت و بالاست، این قانون همبستگی قوی خرید همزمان را اثبات می‌کند.

۳.۶ اصل یکنوایی و الگوریتم A-Priori

چالش محاسباتی اصلی در یافتن مجموعه‌آیتم‌های پرتکرار، رشد نمایی ترکیبات است. اگر N آیتم داشته باشیم، تعداد کل زیرمجموعه‌های ممکن ۲^N است. کلید غلبه بر این انفجار ترکیبیاتی، **اصل یکنوایی**^f است.

قضیه ۱.۶: اصل یکنوایی پشتیبانی

اگر یک مجموعه‌آیتم I پرتکرار باشد، آنگاه تمامی زیرمجموعه‌های آن نیز قطعاً پرتکرار هستند. به بیان عکس (عکس نقیض): اگر مجموعه‌آیتمی نامتکرار باشد، هیچ اُبرمجموعه‌ای از آن نمی‌تواند پرتکرار باشد.

$$J \subseteq I \implies s(I) \leq s(J) \quad (۶.۶)$$

Principle Apriori یا Property Monotonicity^f

۱.۳.۶ مراحل اجرای الگوریتم A-Priori

الگوریتم A-Priori در گذرها^۵ متوالی بر روی داده‌های سبدها عمل می‌کند:

الگوریتم ۱.۶: الگوریتم استاندارد A-Priori []

• گذر اول (Pass 1):

۱. با یک بار خواندن سبدها از روی دیسک، فراوانی هر آیتم منفرد شمرده می‌شود.
۲. آیتم‌هایی که فراوانی آن‌ها $\geq s$ باشد، مجموعه آیتم‌های ۱-تایی پرتکرار (L_1) را تشکیل می‌دهند.
- تولید نامزدهای ۲-تایی (C_2): تنها جفت‌هایی $\{i, j\}$ به عنوان نامزد در نظر گرفته می‌شوند که هر دوی $i \in L_1$ و $j \in L_1$ باشند.
- گذر دوم (Pass 2): سبدها مجدداً از دیسک خوانده می‌شوند. برای هر سبد، تمامی جفت‌های درون آن که در C_2 عضویت دارند پیدا شده و شمارنده آن‌ها یکی افزایش می‌یابد. در پایان گذر دوم، جفت‌های با شمارش $\geq s$ به عنوان L_2 انتخاب می‌شوند.
- گذر k -ام (Pass k): مجموعه نامزدهای C_k از روی L_{k-1} تولید می‌شود، سبدها خوانده شده و L_k به دست می‌آید. این فرایند تا زمانی ادامه می‌یابد که هیچ نامزد جدیدی تولید نشود.

۴.۶ الگوریتم‌های بهینه‌سازی مصرف حافظه

در الگوریتم A-Priori، گلوگاه اصلی حافظه در **گذر دوم** (شمارش تعداد جفت‌های نامزد C_2) رخ می‌دهد. الگوریتم‌های زیر برای حل این گلوگاه ابداع شدند:

۱.۴.۶ الگوریتم پارک-چن-یو (PCY)

مشاهده کلیدی: در گذر اول الگوریتم A-Priori، تنها کاری که انجام می‌شود شمارش آیتم‌های تکی است که حافظه بسیار ناچیزی نیاز دارد؛ در نتیجه بخش اعظم حافظه اصلی کامپیوتر در گذر اول بیکار می‌ماند!

ایده الگوریتم PCY:

۱. در گذر اول، از حافظه آزاد باقیمانده به عنوان یک جدول درهم‌سازی بزرگ از سطرها^۶ استفاده می‌کنیم.
۲. هنگام خواندن هر سبد در گذر اول، علاوه بر شمارش آیتم‌های تکی، به ازای هر جفت $\{i, j\}$ در سبد، مقدار درهم‌سازی $h(i, j)$ را محاسبه کرده و شمارنده آن سطر را یک واحد افزایش می‌دهیم.
۳. در پایان گذر اول، اگر مقدار یک سطر کمتر از آستانه s باشد، امکان ندارد هیچ جفت پرتکراری به آن سطر نگاشت شده باشد.
۴. جدول درهم‌سازی به یک نقشه بیتی^۷ فشرده تبدیل می‌شود (۱ بیت برای هر سطر: ۱ اگر مجموع سطر $\geq s$ باشد، وگرنه ۰).

۵. در گذر دوم: یک جفت $\{i, j\}$ به عنوان نامزد شمارش می‌شود اگر و تنها اگر:

$$i \in L_1 \text{ و } j \in L_1 \text{ و } \text{Bitmap}[h(i, j)] = 1 \quad (۷.۶)$$

این شرط سه‌گانه، تعداد جفت‌های نامزد در گذر دوم را به شدت کاهش می‌دهد.

^۵ Passes
^۶ Hash Table of Buckets
^۷ Bitmap

۲.۴.۶ الگوریتم‌های چندمرحله‌ای و چنددرهم‌سازی (Multistage & Multihash)

- **الگوریتم چندمرحله‌ای (Multistage):** یک گذر میانی (گذر ۵.۱) اضافه می‌کند که در آن، تنها جفت‌هایی که از فیلتر سطل اول عبور کرده‌اند با یک تابع درهم‌سازی مستقل دوم هاش می‌شوند؛ بدین ترتیب نقشه بیتی دوم تولید شده و تعداد نامزدهای گذر نهایی باز هم کاهش می‌یابد.
- **الگوریتم چنددرهم‌سازی (Multihash):** در همان گذر اول، حافظه را به دو یا چند جدول درهم‌سازی مستقل با دو تابع هاش مجزا تقسیم می‌کند.

۵.۶ الگوریتم‌های با تعداد گذر محدود (یک یا دو گذره)

در کلان‌داده، خواندن مکرر ترابایت‌ها داده از دیسک زمان‌بر است. الگوریتم‌های زیر یافتن مجموعه‌آیتم‌های پرتکرار را در ۱ یا ۲ گذر میسر می‌سازند:

۱.۵.۶ الگوریتم ساواسره-امینسکی-ناواته (SON)

الگوریتم SON ساختاری کاملاً موازی دارد و بهترین الگوریتم برای پیاده‌سازی در **MapReduce** است:

الگوریتم ۲.۶: الگوریتم SON در MapReduce

۱. گذر اول (گام MapReduce اول):

- داده‌ها به m قطعه مساوی (Chunk) تقسیم می‌شوند.
 - هر میر یک قطعه را خوانده و با استفاده از الگوریتم A-Priori یا PCY، مجموعه‌آیتم‌های پرتکرار محلی با آستانه متناسب s/m را پیدا می‌کند.
 - **لم بنیادین:** هر مجموعه‌آیتمی که در کل داده‌ها پرتکرار باشد، حداقل در یکی از m قطعه باید دارای پشتیبانی $s/m \geq$ باشد.
 - کاهنده اجتماع تمامی مجموعه‌آیتم‌های پرتکرار محلی را به عنوان نامزدهای جهانی منتشر می‌سازد.
۲. **گذر دوم (گام MapReduce دوم):** میرها سبدها را خوانده و پشتیبانی دقیق نامزدهای جهانی را شمارش می‌کنند؛ کاهنده نامزدهایی را که مجموعاً $s \geq$ هستند به عنوان پاسخ قطعی و نهایی برمی‌گزیند.

ویژگی منحصر به فرد SON: دقیقاً در ۲ گذر اجرا می‌شود و هیچ منفی کاذب یا مثبت کاذبی تولید نمی‌کند!

۲.۵.۶ الگوریتم توینن (Toivonen's Algorithm) و مرز منفی

الگوریتم Toivonen با یک گذر بر روی نمونه و یک گذر بر روی کل داده‌ها عمل می‌کند:

۱. نمونه‌ای تصادفی با اندازه کسری p از سبدها برداشته می‌شود.
۲. مجموعه‌آیتم‌های پرتکرار در نمونه با آستانه‌ای اندکی کمتر از $p \cdot s$ استخراج می‌گردند.
۳. **مرز منفی (Negative Border)** تشکیل می‌شود: مجموعه‌آیتم‌هایی که خود در نمونه پرتکرار نشده‌اند، اما تمامی زیرمجموعه‌های بلافاصل آن‌ها پرتکرار بوده‌اند.
۴. در گذر دوم بر روی کل داده‌های اصلی، پشتیبانی تمامی آیتم‌های پرتکرار نمونه و تمامی آیتم‌های مرز منفی به طور هم‌زمان شمرده می‌شود.
۵. **شرط موفقیت:** اگر هیچ‌یک از اعضای مرز منفی در داده‌های اصلی به آستانه s نرسند، نتایج نمونه برای داده‌های اصلی کاملاً دقیق و بدون خطا است! (اگر عضوی از مرز منفی پرتکرار شود، آزمون باطل شده و با نمونه جدید تکرار می‌گردد).

نکته امتحانی و کلیدی

نکات امتحانی پرکاربرد:

۱. **فرمول آدرس‌دهی ماتریس مثلثی:** برای ذخیره شمارنده‌های جفت‌های نامزد در یک آرایه یک‌بعدی فشرده (بدون اتلاف فضای قطر و بخش بالایی)، درایه (i, j) با شرط $i < j$ در خانه زیر ذخیره می‌شود:

$$k = (i - 1) \left(n - \frac{i}{2} \right) + j - i \quad (۸.۶)$$

مقایسه روش ماتریس مثلثی در برابر سه‌تایی‌ها (Triples): هر خانه ماتریس مثلثی ۴ بایت (یک عدد صحیح) فضا می‌گیرد؛ اما روش سه‌تایی‌ها برای هر جفت نیازمند ۱۲ بایت $((i, j, \text{count}))$ است. بنابراین اگر بیش از یک سوم کل جفت‌های ممکن نامزد باشند، ماتریس مثلثی فشرده‌تر است؛ اما اگر جفت‌های نامزد بسیار تنگ باشند، روش سه‌تایی‌ها کاراتر خواهد بود.

۲. **اثبات صفر بودن منفی‌های کاذب در الگوریتم SON:** اگر مجموعه‌آیتم I در کل داده‌ها دارای پشتیبانی حداقل s باشد، غیرممکن است که در تمام m قطعه دارای پشتیبانی کمتر از s/m باشد؛ زیرا در این صورت مجموع کل پشتیبانی اکیداً کمتر از $s \times (s/m) = m \times (s/m)$ می‌شد که تناقض است. بنابراین I قطعاً حداقل در یک قطعه کشف شده و هیچ آیتم پرتکراری از دست نمی‌رود.

۳. **نقش حیاتی مرز منفی در الگوریتم Toivonen:** اگر کوچک‌ترین آیتمی که در کل داده‌ها به حد نصاب رسیده اما در نمونه کشف نشده وجود داشته باشد، طبق اصل یکنوایی تمام زیرمجموعه‌های بلافاصل آن باید در نمونه کشف شده باشند؛ پس آن آیتم حتماً عضوی از مرز منفی است و در گذر دوم شمرده می‌شود.

جمع‌بندی فصل ششم

- مدل سبد بازار همبستگی‌های هم‌نشینی را از طریق معیارهای پشتیبانی، اطمینان و بهره ارزیابی می‌کند.
- اصل یکنوایی با حذف اَبَرمجموعه‌های اقلام نامتکرار، مانع انفجار ترکیبیاتی می‌شود.
- الگوریتم PCY با به کارگیری حافظه بلااستفاده برای سطل‌های درهم‌سازی، گلوگاه گذر دوم را مرتفع می‌سازد.
- الگوریتم SON چارچوبی بهینه برای موازی‌سازی کاوش در بستر MapReduce تنها در دو گذر است.
- الگوریتم Toivonen به کمک مرز منفی، بررسی صحت نتایج نمونه‌گیری را تضمین می‌نماید.